



# Mobility-Aware Task Scheduling for Fog-Enabled Smart Hospitals

GUNA VARDHAN BYRAJU(AP23110010744)

A Research Project of SRM University  
AP

**SRM University AP**  
Providing Unique Learning Experience

# The Mobility Challenge: Latency and Resource

**Mobility** **Challenge** **in** smart hospitals frequently encounter connectivity and resource allocation issues as they navigate dynamic environments. This mobility introduces critical challenges to real-time task processing.



## Robot Movement & Task Stagnation

As robots traverse hospital zones, tasks may remain tethered to suboptimal or disconnected fog nodes, leading to processing delays.



## Critical Deadlines Missed

Time-sensitive alerts, such as sepsis detection, demand sub-second response times. Mobility-induced latency can critically compromise these deadlines.



## Inefficient Energy Consumption

Processing tasks on distant fog nodes or the cloud due to poor local connectivity results in increased energy expenditure for data transmission.



# Mobility-Aware Scheduling Algorithm: GNN & Meta-DRL

Our proposed algorithm integrates Graph Neural Networks (GNNs) and Meta-Deep Reinforcement Learning (Meta-DRL) to adaptively schedule tasks for mobile fog robots, mitigating latency and optimising resource utilisation.

## 1. Real-time Movement Tracking (GNN Layer)

The GNN continuously updates the dynamic graph representing robot-fog node proximity every 0.1 seconds. Edge strengths reflect connection quality, weakening as robots move away.

```
python gnn.update_edge(robot=ICU_Robot, fog_node=ICU_Fog,
new_strength=0.2)
```

## 2. Adaptive Task Prioritisation (Meta-DRL Policy)

The Meta-DRL policy assigns task priorities and offloading decisions based on real-time context, including deadlines and fog node load.

- Rule 1: If task.deadline < 1s: offload(nearest\_fog)
- Rule 2: If fog\_node.load > 80%: offload(cloud, energy\_penalty=-5)
- Rule 3: If robot.speed > 0: predict\_next\_node(gnn)

# Mobility-Aware Scheduling Algorithm: Meta-Learning for Robustness

The Meta-DRL component of our algorithm is designed for rapid adaptation to unforeseen scenarios and dynamic hospital environments, ensuring robust task scheduling.



## Meta-Training across Diverse Layouts

The model is meta-trained across over 10 distinct hospital layouts, learning generalisable policies for varying operational conditions.



## Adaptation to New Robot Paths

Pre-trained policies enable swift adaptation to novel robot trajectories, such as movements from the ER to Radiology.



## Dynamic Obstacle Handling

The system can autonomously adjust scheduling in response to dynamic obstacles, like blocked hallways or temporary zone closures.

# Pseudocode: Adaptive Scheduling Task

```
def schedule_task(robot, task, fog_nodes, cloud_node):  
    # Step 1: GNN updates fog node connections based on robot  
    movement  
    gnn.update_graph(robot, fog_nodes) # Updates edge weights every  
    0.1s  
  
    # Step 2: Meta-DRL makes offloading decision  
    if task.deadline < 1.0: # Emergency task (e.g., sepsis alert)  
        best_node = gnn.find_nearest_node(robot) # Prioritize lowest  
        latency  
        return best_node, "FOG_EMERGENCY"  
  
    elif max([node.load for node in fog_nodes]) > 0.8: # Fog nodes  
    overloaded  
        return cloud_node, "CLOUD_SAFE" # Offload to cloud (penalize  
        energy)  
  
    else: # Normal task, use Meta-DRL's learned policy  
        best_node = meta_drl.predict(robot, task, fog_nodes)  
        return best_node, "FOG_OPTIMAL"
```



# Case Study: Sepsis Alert — Minimising Critical Latency

This scenario demonstrates the critical performance advantage of our method in handling time-sensitive medical alerts compared to existing approaches.

|                        |                                                                          |                                                                          |
|------------------------|--------------------------------------------------------------------------|--------------------------------------------------------------------------|
| Scenario               | Robot in ICU detects sepsis, begins moving to ER.<br>Deadline: 1 second. | Robot in ICU detects sepsis, begins moving to ER.<br>Deadline: 1 second. |
| 1. Task Initialisation | Assigned to ICU Fog                                                      | Assigned to ICU Fog                                                      |
| 2. Robot Mobility      | Still uses ICU Fog (high latency due to distance)                        | GNN detects movement → Switches to ER Fog<br>(proactive offload)         |
| 3. Result              | 2.5s (FAILURE to meet deadline)                                          | 0.8s (SUCCESS in meeting deadline)                                       |

The integration of GNN for real-time location tracking and Meta-DRL for immediate, adaptive decision-making (specifically the emergency rule) ensures critical tasks are handled with minimal latency, directly impacting patient safety outcomes.

# Case Study: Routine Vitals Check — Optimising System Stability

This case illustrates our method's ability to balance energy efficiency with overall system stability, contrasting with approaches that risk local fog overload.

## Scenario:

- Robot in Radiology sends hourly vitals data.
- Deadline: 5 minutes (less critical than sepsis alert).

| Target   | Local fog (risky if overloaded) | Cloud (if fog load >80%)                   |
|----------|---------------------------------|--------------------------------------------|
| Energy   | 15J (fog)                       | 18J (cloud, but saves fog for emergencies) |
| Fog Load | 90% CPU                         | 40% CPU                                    |

While our method incurs slightly higher energy consumption by offloading routine tasks to the cloud, this proactive decision prevents local fog node overload. This strategy ensures critical, time-sensitive tasks can always be processed locally with minimal latency, thereby enhancing overall system stability and reliability for emergency situations.

